

Reviewer Pack — Minimal Order of Quaternionic Kähler Manifolds and Circuit M...

Entry c3a93f0dc63d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 08:35:44 UTC

Minimal Order of Quaternionic Kähler Manifolds and Circuit Monotone Width

Entry ID: c3a93f0dc63d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 08:35:44 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Geometry (Quaternionic Kähler Manifolds) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every boolean circuit C with n inputs, the minimal dimension d of a quaternionic Kähler manifold supporting C is upper-bounded by $O(n^{2/3})$ and lower-bounded by $\Omega(n^{1/3})$.

Rationale (proposer's reasoning):

Quaternionic Kähler manifolds have been studied in differential geometry but are rarely applied to complexity theory. This conjecture bridges the gap by proposing a connection between the geometric property of these manifolds and circuit complexity. The potential link arises from the fact that quaternionic Kähler manifolds are associated with symplectic structures, which may offer insights into the geometry of boolean functions.

Taxonomy category: Quaternionic_Kahler (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9703811499eb404c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the ratio of the minimal dimension d of a quaternionic Kähler manifold to the monotone width $w_m(C)$ of the corresponding circuit C satisfies $O(n^{2/3}) \leq d/w_m(C)/n^{2/3} \leq \Omega(n^{1/3})$, for all circuits with n inputs ($n \leq 40$), across 30 random seeds. The conjecture is falsified if this ratio exceeds these bounds for any seed or circuit size.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quaternionic Kähler Manifolds" AND "circuit monotone width" - "minimal dimension" quaternionic Kähler manifold "circuit complexity" - "order of quaternionic Kähler manifolds" related to boolean circuits

Top relevant hits considered: - [<http://arxiv.org/abs/2206.07946v2>] Hermitian structures on a class of quaternionic Kähler manifolds - [<http://arxiv.org/abs/1607.07232v2>] Completeness of projective special Kähler and quaternionic Kähler manifolds - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/nlin/0208039v1>] Effects of RF Stimulus and Negative Feedback on Nonlinear Circuits - [<http://arxiv.org/abs/2405.20178v2>] Non-intrusive data-driven model order reduction for circuits based on Hammerstein architectures

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return ['0']
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return [f'({left[0]} + {right[0]})', f'({left[0]} * {right[0]})']

    def evaluate_circuit(circuit, assignment):
        stack = []
        for token in circuit:
            if token.isdigit():
                stack.append(assignment[token])
            elif token == '+':
                b = stack.pop()
```

```

        a = stack.pop()
        stack.append(a + b)
    elif token == '*':
        b = stack.pop()
        a = stack.pop()
        stack.append(a * b)
    return stack[0]

def monotone_width(circuit):
    if not circuit:
        return 0
    max_width = 1
    current_width = 1
    for i in range(1, len(circuit)):
        if circuit[i] == '(':
            current_width += 1
        elif circuit[i] == ')':
            current_width -= 1
        max_width = max(max_width, current_width)
    return max_width

def quaternionic_kahler_dimension(n):
    # Simplified approximation for demonstration purposes
    return Fraction(n ** (2 / 3))

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_circuit(n)
    assignment = {str(i): random.randint(1, 10) for i in range(n)}
    value = evaluate_circuit(circuit, assignment)
    width = monotone_width(circuit)
    d = quaternionic_kahler_dimension(n)

    if width == 0:
        continue

    ratio = d / width
    results.append({
        "n": n,
        "value": value,
        "width": width,
        "d": d,
        "ratio": ratio
    })

total_ratio = sum(result['ratio'] for result in results)
avg_ratio = total_ratio / len(results) if results else 0

return {
    "metric_name": "Ratio of Quaternionic Kähler Dimension to Monotone Width",
    "metric_value": avg_ratio,
    "instances_tested": len(results),
    "n_max": max(result['n'] for result in results),
    "conjecture_holds": all(0.5 * n ** (1 / 3) <= ratio <= 2 * n ** (2 / 3) for result in results)
}

```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    avg_ratio = sum(result['metric_value'] for result in results) / len(results)
    support_fraction = sum(result['conjecture_holds'] for result in results) / len(results)

    if all(result['conjecture_holds'] for result in results):
        print(f"RESULT: SUPPORTED mean={avg_ratio} std=0 support_fraction={support_fraction}")
    elif any(not result['conjecture_holds'] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample='ratio_outside_bounds' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7fc457c8.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7fc457c8.py", line 67, in run_trial
    value = evaluate_circuit(circuit, assignment)
             ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7fc457c8.py", line 42, in evaluate_circuit
    return stack[0]
           ~~~~~^~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes without crashing and produces the expected results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15145
2	preregistration	ollama_remote	glm4:latest	0	0	10078
3	novelty	ollama_remote	glm4:latest	0	0	8671
4	novelty	ollama_remote	glm4:latest	0	0	10156
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15306
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7797
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11420
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10152
9	judge	ollama_remote	glm4:latest	0	0	15239

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 103964 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c3a93f0dc63d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c3a93f0dc63d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c3a93f0dc63d.tar.gz (if generated)